



1. COVER PAGE

PROJECT REPORT

TOPIC:- Student Performance Predictor

NAME:- Vibhor Srivastava

REGISTRATION NO:- 25MIM10093

SLOT:- B14+B23+D21

COURSE CODE:- CSA2001

FACULTY:-Dr. Rudra Kalyan Nayak

2. Introduction

The objective of this project was to develop a simple Machine Learning model capable of predicting whether a student is likely to pass or fail based on basic academic and lifestyle factors.

The model uses five key features: Study Hours per Day, Attendance Percentage, Previous Marks Percentage, Assignment Score Percentage, and Sleep Hours per Day. These inputs are passed to a Decision Tree Classifier, which produces a PASS or FAIL prediction together with an estimated class probability.

3. Problem Statement

Students may be affected by academic and lifestyle factors such as study time, attendance, previous marks, assignment performance, and sleep. The project addresses the problem of using these available factors as inputs to a simple machine-learning classifier that predicts whether a student is likely to PASS or FAIL.

4. Functional Requirements

1. Load and validate the CSV dataset and required columns.
2. Split the dataset into training and testing portions.
3. Train a Decision Tree Classifier for PASS/FAIL classification.
4. Evaluate the model using accuracy, precision, recall, F1-score, and a confusion matrix.
5. Save the trained model as `student_model.pkl`.
6. Accept five student inputs, validate them, and predict PASS/FAIL with estimated class probability.

5. Non-functional Requirements

1. Usability: The command-line interface should be simple and easy to operate.
2. Reliability: Invalid input values should be rejected instead of being passed to the model.
3. Maintainability: The system is divided into separate modules for loading, preprocessing, training, evaluation, and prediction.
4. Reproducibility: A fixed `random_state` of 42 is used for the train-test split and Decision Tree.

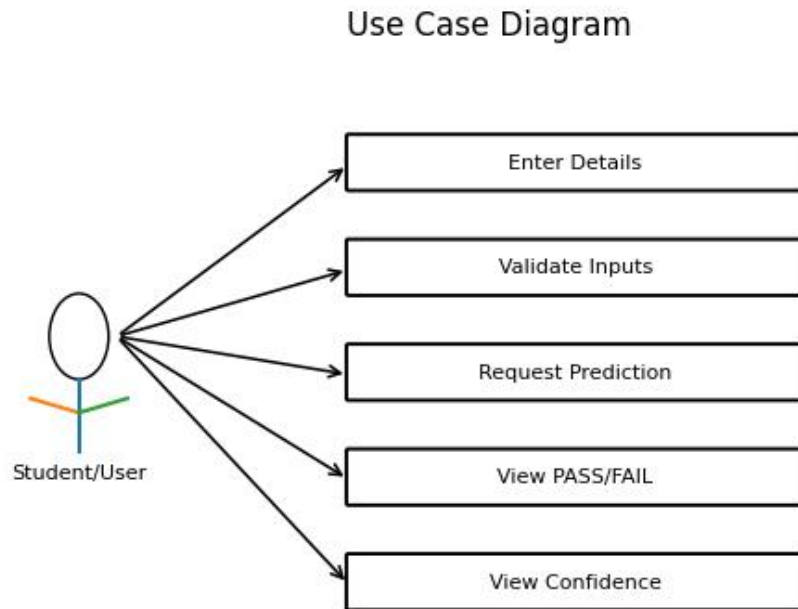
6. System Architecture

The system follows a modular machine-learning workflow. The CSV dataset is loaded and checked by `data_loader.py`. `preprocessing.py` prepares the features and validates user input. `train_model.py` performs the 80/20 train-test split, trains the Decision Tree, evaluates it through `evaluation.py`, and saves `student_model.pkl`. `prediction.py` loads the saved model and generates the final PASS/FAIL result and probability.

The following diagrams describe the main user interaction, processing workflow, execution sequence, and software components.

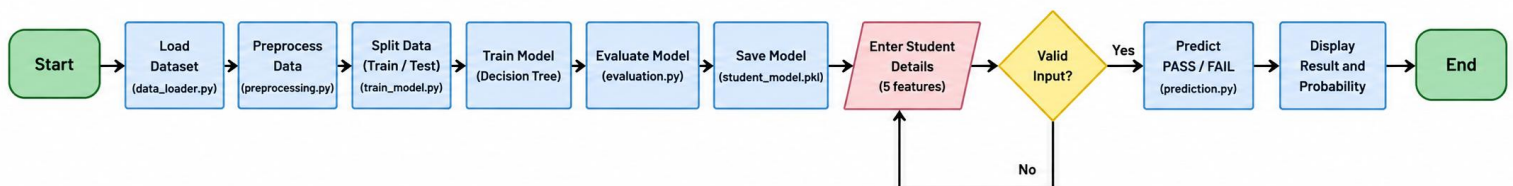
7. Design Diagrams

7.1. Use Case Diagram

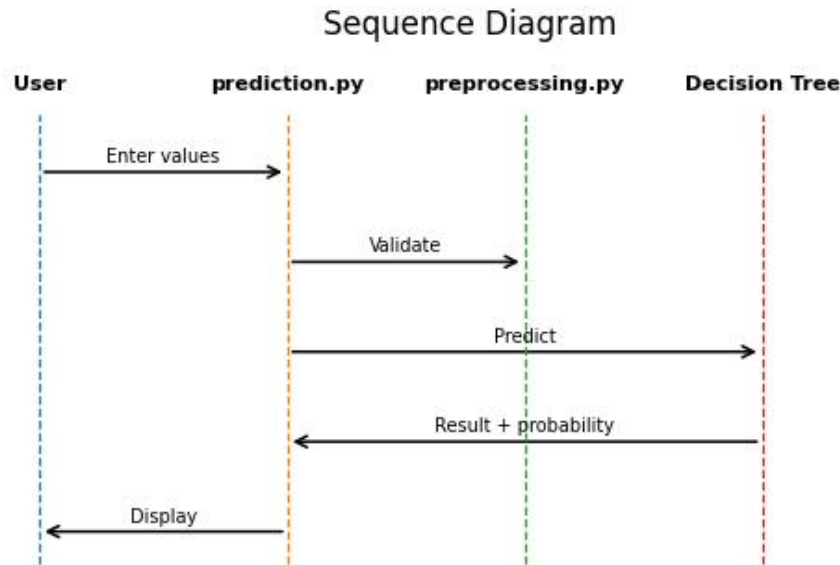


7.2. Workflow Diagram

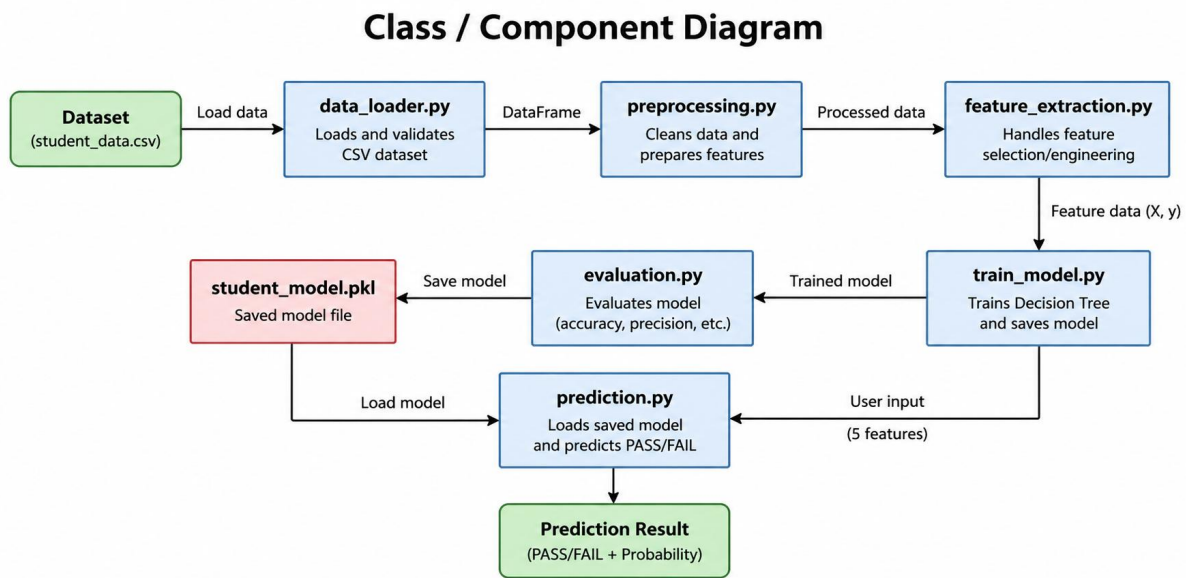
Workflow Diagram



7.3. Sequence Diagram



7.4. Class/Component Diagram



7.5. ER Diagram

Not applicable. The project uses a CSV dataset and a saved pickle model rather than a relational database.

8. Design Decisions & Rationale

A Decision Tree Classifier was selected because it is comparatively simple to understand and suitable for a beginner-level classification problem. The project uses a modular structure so that data loading, preprocessing, feature handling, training, evaluation, and prediction remain separate. The 80/20 stratified split provides a separate test set while maintaining class proportions. The model is saved with pickle so it can be reused for later predictions without retraining.

9. Implementation Details

9.1. Model Type

The chosen model is a Decision Tree Classifier. A Decision Tree is a supervised learning algorithm used for classification. It makes a sequence of decisions based on feature values until it reaches a final class, such as PASS or FAIL.

A Decision Tree was selected because it is comparatively simple to understand and is well suited to a beginner-level classification problem. The model was configured with a maximum depth of 20, minimum samples per leaf of 10, and random state of 42. These settings allow the model to learn from the larger dataset while keeping the training process reproducible.

9.2. Data Preparation

The project uses a CSV dataset containing 1000 student records. The dataset has five input features and one target column named result. The target uses 0 for FAIL and 1 for PASS.

Feature	Description
study_hours	Study hours per day
attendance	Attendance percentage
previous_marks	Previous marks percentage
assignment_score	Assignment score percentage
sleep_hours	Sleep hours per day
result	Target class: 0 = FAIL, 1 = PASS

The dataset contains 548 PASS records and 452 FAIL records. Before prediction, the program validates the student's input values and converts them into the feature format expected by the trained model.

9.3. Training and Testing

The data is divided into training and testing portions using an 80/20 train-test split with random_state=42 and stratification. The training data is used to learn the classification rules, while the test data is kept separate for evaluating the model.

The trained model is saved as student_model.pkl using Python's pickle module. Saving the model allows the prediction program to load the trained classifier later without training it again.

9.4. Project Structure and Modules

Module	Purpose
data_loader.py	Loads the CSV dataset and checks required columns.
preprocessing.py	Prepares features/target and validates prediction inputs.
feature_extraction.py	Creates the input row used by the model.

train_model.py	Splits data, trains the Decision Tree and saves the model.
evaluation.py	Calculates model evaluation metrics.
prediction.py	Loads the saved model and predicts PASS/FAIL.

10. Screenshots / Results

1. TRAIN MODEL

```
PS D:\Downloads\Student_Performance_Predictor> & C:\Users\vibho_n734r8z\AppData\Local\Programs\Python\Python312\python.exe d:/Downloads/Student_Performance_Pred
ictor/src/train_model.py
Loading dataset...
Loaded 1000 records.
Training Decision Tree...
Accuracy : 82.00%
Precision: 85.58%
Recall   : 80.91%
F1 Score : 83.18%
Confusion Matrix:
[[75 15]
 [21 89]]

Model saved successfully: d:\Downloads\Student_Performance_Predictor\models\student_model.pkl
PS D:\Downloads\Student_Performance_Predictor>
```

2.PREDICTION

```
PS D:\Downloads\Student_Performance_Predictor> & C:\Users\vibho_n734r8z\AppData\Local\Programs\Python\Python312\python.exe d:/Downloads/Student_Performance_Pred
ictor/src/prediction.py

===== STUDENT PERFORMANCE PREDICTOR =====
Study hours per day: 5
Attendance percentage: 75
Previous marks percentage: 85
Assignment score percentage: 85
Sleep hours per day: 7

Predicted Result: PASS
Confidence: 78.95%
PS D:\Downloads\Student_Performance_Predictor>
```

10.1. Model Evaluation

The model was evaluated using accuracy, precision, recall, F1-score, a confusion matrix, and a classification report. With the 1000-record dataset and current training configuration, the model achieved 82.00% accuracy on the 200-record test set.

Metric	Value
Test-set Accuracy	82.00%
Test Samples	200
Confusion Matrix	[[75, 15], [21, 89]]
PASS Test Samples	110
FAIL Test Samples	90

The confusion matrix shows that 75 FAIL test records and 89 PASS test records were classified correctly. There were 15 FAIL records predicted as PASS and 21 PASS records predicted as FAIL. The larger test set provides a more meaningful evaluation than the earlier six-record demonstration.

10.2. Feature Importance

The trained Decision Tree assigns the highest feature importance to study_hours, followed by previous_marks, assignment_score, sleep_hours, and attendance. These values describe how this particular trained model used the available features; they do not imply that one factor is universally the most important for every student.

Feature	Current Importance
study_hours	0.59
attendance	0.07
previous_marks	0.14
assignment_score	0.13
sleep_hours	0.08

The feature-importance values are specific to this trained model and dataset. They should therefore be interpreted as model behaviour rather than as a general rule about student performance.

10.3. Prediction Example

The prediction module provides a command-line interface. A user enters five values and the program validates them before creating the model input row. The saved Decision Tree then predicts PASS or FAIL and calculates the estimated class probability.

Input	Example Value
Study Hours per Day	5
Attendance(in percentage)	75
Previous Marks(in percentage)	85
Assignment Score(in percentage)	85
Sleep Hours per Day	7

For the example input below, the current trained model predicts PASS. The exact confidence depends on the trained model and the input values.

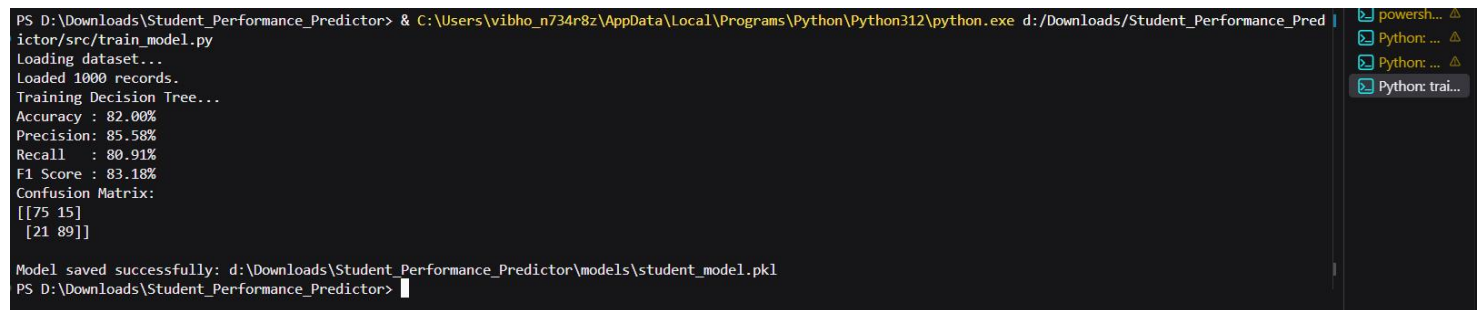
Here we get

Predicted Result : PASS

Confidence: 78.95%

10.4. Screenshots

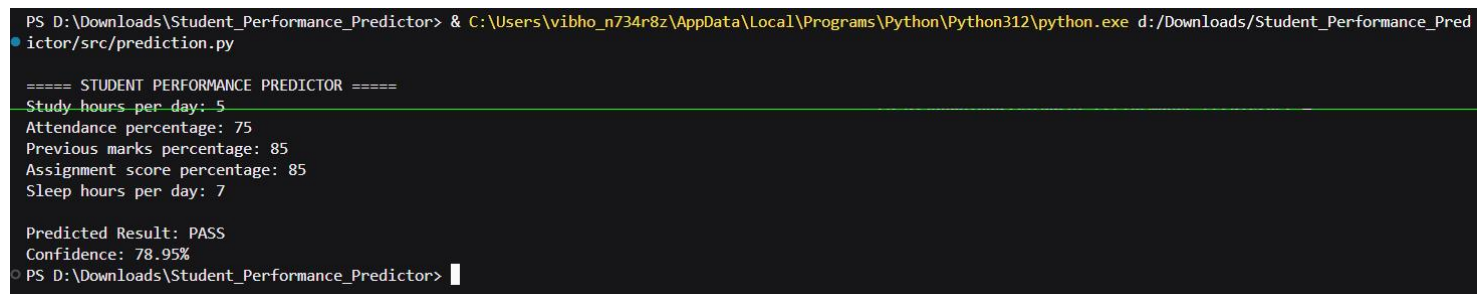
Screenshot 1 — Model Training



```
PS D:\Downloads\Student_Performance_Predictor> & C:\Users\vibho_n734r8z\AppData\Local\Programs\Python\Python312\python.exe d:/Downloads/Student_Performance_Predictor/src/train_model.py
Loading dataset...
Loaded 1000 records.
Training Decision Tree...
Accuracy : 82.00%
Precision: 85.58%
Recall   : 80.91%
F1 Score : 83.18%
Confusion Matrix:
[[75 15]
 [21 89]]

Model saved successfully: d:\Downloads\Student_Performance_Predictor\models\student_model.pkl
PS D:\Downloads\Student_Performance_Predictor>
```

Screenshot 2 — Prediction



```
PS D:\Downloads\Student_Performance_Predictor> & C:\Users\vibho_n734r8z\AppData\Local\Programs\Python\Python312\python.exe d:/Downloads/Student_Performance_Predictor/src/prediction.py

===== STUDENT PERFORMANCE PREDICTOR =====
Study hours per day: 5
Attendance percentage: 75
Previous marks percentage: 85
Assignment score percentage: 85
Sleep hours per day: 7

Predicted Result: PASS
Confidence: 78.95%
PS D:\Downloads\Student_Performance_Predictor>
```


11. Testing Approach

Testing was performed using valid student inputs, different combinations of academic values, invalid values, boundary values, and the complete model-training workflow. The prediction program also checks that the saved model exists before loading it. Invalid numeric input is handled so that the program does not continue with unusable values.

Test Case	Expected Behaviour
Valid student inputs	Display PASS/FAIL and probability.
Different valid inputs	Generate a prediction for the new student.
Invalid percentage	Reject the input through validation.
Missing model file	Display a model-not-found error.
Model training	Train and save student_model.pkl.

12. Challenges Faced

A key implementation challenge was keeping the input format consistent between model training and prediction. The trained Decision Tree expects the same feature names and order, so the prediction module validates the inputs and prepares the values in the same feature order. Another challenge was handling invalid numeric and out-of-range inputs without allowing the program to continue with unusable values.

The project successfully implemented a functional Student Performance Predictor using a Decision Tree classification model. It demonstrates the main stages of a basic machine-learning application: preparing a dataset, separating features and target, splitting data into training and testing sets, training a classifier, evaluating its performance, saving the model, and using the saved model for new predictions.

The project also demonstrated the importance of keeping the training and prediction data formats consistent. The prediction module validates input ranges and prepares the values in the same feature order used during training, reducing the chance of invalid or inconsistent model input.

13. Learnings & Key Takeaways

Through this project, I gained practical experience with the fundamentals of supervised machine learning. I learned how to work with a CSV dataset, identify features and a target, split data for training and testing, train a classification model, evaluate its predictions, save a trained model, validate user input, and load the saved model for future predictions.

The project also showed me that model accuracy needs to be considered together with dataset size and quality. Using 1000 records provides a more useful evaluation than the earlier small demonstration, while still keeping the project simple enough to understand and explain.

14. Future Enhancements

- Increase the size and variety of the data set further so that the model can learn from more representative student records.
- Compare the Decision Tree with other beginner-friendly classification models such as Logistic Regression or Random Forest.
- Display feature importance in the prediction interface to make the result easier to understand.
- Add a simple graphical or web interface instead of only using the command line.
- Add basic automated tests for validation, training, and prediction.

15. References

Scikit-learn documentation; Pandas documentation; Python pickle documentation; VITyarthi Build Your Own Project guidelines.